

FakeNewsChecker
Object Design Document
Versione 1.0



**FakeNews
Checker**

Data: 20/12/2025

Progetto: FakeNewsChecker	Versione: 1.0
Documento: Object Design Document	Data: 20/12/2025

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
De Simone Federica	0512120472
Tirelli Giuseppe	0512120367
Falasca Swami	0512119323

Scritto da:	Swami Falasca, De Simone Federica, Tirelli Giuseppe
--------------------	---

Revision History

Data	Versione	Descrizione	Autore
20/12/2025	1.0	Prima versione dell'Object Design Document	Swami Falasca, De Simone Federica, Tirelli Giuseppe

Indice

1. INTRODUZIONE	4
1.1. OBJECT DESIGN TRADE-OFFS	4
1.1.1. Tempo di rilascio VS Funzionalità	4
1.1.2. Velocità VS Memoria	4
1.1.3. Costruire VS Comprare	4
1.2. OFF-THE SHELF COMPONENTS	4
1.3. INTERFACE DOCUMENTATION GUIDELINES	5
1.4. DEFINITIONS, ACRONYMS AND ABBREVIATIONS	6
1.5. REFERENCES	6
2. PACKAGES	6
2.1. SUBSYSTEM DECOMPOSITION	6
2.1.1. Interface	7
2.1.2. Application logic	7
2.1.3. Storage	8
3. CLASS INTERFACES	8
3.1. AccountUserDao	8
3.2. NotiziaDao	10
3.3. SegnalazioneDao	11
3.4. StoricoSegnalazioneDao	12

1. INTRODUZIONE

1.1. *OBJECT DESIGN TRADE-OFFS*

1.1.1. *Tempo di rilascio VS Funzionalità*

Per garantire un sistema completo e affidabile per i clienti, abbiamo scelto di privilegiare la completezza delle funzionalità rispetto al tempo di rilascio.

1.1.2. *Velocità VS Memoria*

Utilizzare più memoria per ottimizzare l'accesso veloce ai dati può aumentare le prestazioni, anche se potrebbe richiedere risorse aggiuntive.

1.1.3. *Costruire VS Comprare*

Nonostante l'uso di software già disponibile offra vantaggi come l'accesso a funzionalità pronte e un carico di lavoro ridotto per gli sviluppatori, si è preferito costruire la maggior parte del Sistema da zero, ricorrendo a componenti esterne solo in circostanze specifiche.

Trade-Off	
Tempo di rilascio	Funzionalità
Velocità	Memoria
Costruire	Comprare

1.2. *OFF-THE SHELF COMPONENTS*

Il Sistema utilizzerà i seguenti componenti off-the-shelf:

1. **JQuery**: framework ampiamente utilizzato per semplificare la scrittura di codice JavaScript.
2. **MySQL**: sistema open source di gestione di database relazionali SQL.
3. **MySQL Connector/J (JDBC driver)**: driver JDBC ufficiale per la comunicazione tra Java e MySQL.
4. **Tomcat**: Web Server e application container per applicazioni Java.

1.3. INTERFACE DOCUMENTATION GUIDELINES

Queste linee guida definiscono una serie di norme che il team di sviluppo deve rispettare durante la progettazione del sistema. Il progetto FakeNews Checker è sviluppato utilizzando l'ambiente EclipseIDE e presenta la seguente organizzazione:

- **Package:** i nomi dei pacchetti devono essere espressi esclusivamente in caratteri minuscoli.
- **Classi:** la denominazione delle classi deve essere significativa e aderire alla convenzione UpperCamelCase.
- **Interfacce:** anche le interfacce seguono la convenzione UpperCamelCase per la scelta del nome.
- **Metodi:** i nomi dei metodi devono conformarsi allo stile camelCase.
- **Variabili:** la scelta del nome per le variabili deve essere esplicativa. Sono da evitare denominazioni composte da singole lettere, fatta eccezione per variabili temporanee.

Indentazione:

- Le istruzioni contenute in un blocco (ad esempio, un ciclo for) devono essere indentate di un livello rispetto all'istruzione composta.
- La parentesi graffa di apertura deve essere posizionata alla fine della riga dell'istruzione composta.
- La parentesi graffa di chiusura deve allinearsi orizzontalmente con il livello di indentazione dell'istruzione composta.

HTML: per lo sviluppo HTML è stato adottato il riferimento:

https://www.w3schools.com/html/html5_syntax.asp

CSS: gli stili non in-line devono essere definiti esclusivamente in fogli di stile esterni.

JavaScript:

- Il codice JavaScript deve essere organizzato in file dedicati.
- Per l'impaginazione e la denominazione degli elementi, il codice JavaScript deve uniformarsi alle convenzioni stabilite per Java.

Database SQL: la denominazione di tabelle e campi deve rispettare queste regole:

- Utilizzare esclusivamente lettere minuscole. In caso di nomi composti da più parole, separare i termini mediante underscore (_).

1.4. DEFINITIONS, ACRONYMS AND ABBREVIATIONS

- JQuery: framework utilizzato per semplificare la scrittura di codice JavaScript
- MySQL: sistema di gestione di database relazionali SQL
- MySQL Connector/J (JDBC driver): driver JDBC ufficiale che consente e semplifica la comunicazione tra applicazioni Java e il database MySQL.
- SQL: Structured Query Language
- JDBC: Java DataBase Connectivity
- HTML: HyperText Markup Language
- CSS: Cascading Style Sheets

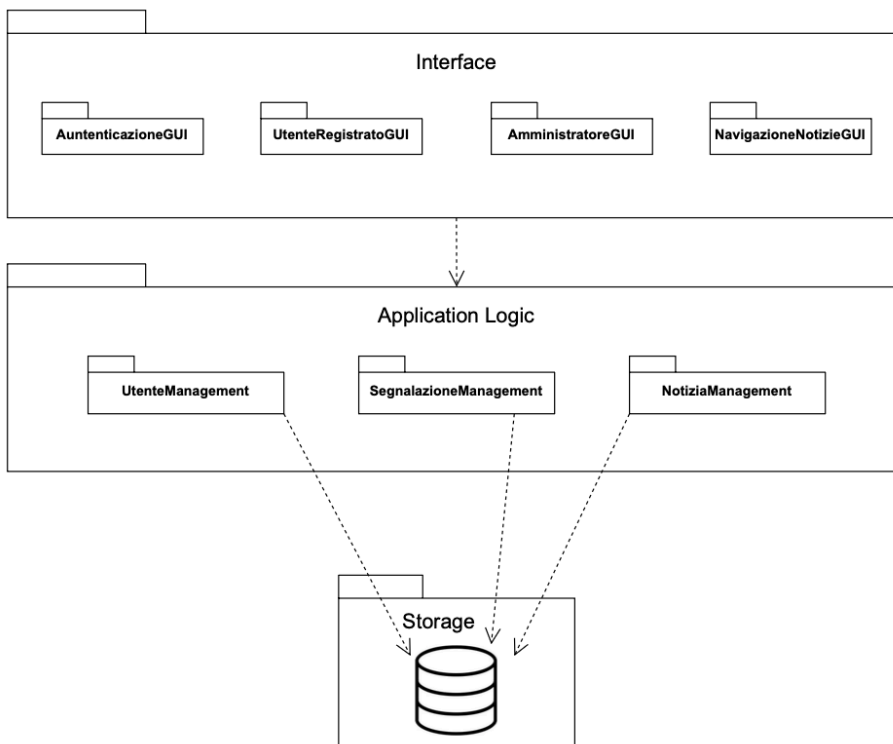
1.5. REFERENCES

Per stilare questo documento, è stato preso come riferimento il libro di testo “Object-Oriented Software Engineering Using UML, Patterns and Java: Third Edition, di Bernd Bruegge ed Allen H. Dutoit”.

2. PACKAGES

In questa parte viene presentata l'organizzazione in package del Sistema, come definito nel progetto di System Design. La ripartizione è conseguenza delle scelte architetturali effettuate e evidenzia l'aderenza alle strutture delle directory di Maven.

2.1. SUBSYSTEM DECOMPOSITION



2.1.1. Interface

La struttura del package si articola nei seguenti sub-package e classi:

- Package AutenticazioneGUI
 - InserisciEmail
 - Login
 - Registrati
- Package UtenteRegistratoGUI
 - AreaUtente
 - ConfermaSegnalazione
 - StoricoSegnalazioni
- Package AmministratoreGUI
 - AmministratoreVerifiche
 - AggiornaStato
 - AmministratoreTecnico
 - DashboardTecnica
- Package NavigazioneNotiziaGUI
 - RicercaNotizie

Sono inoltre definite le interfacce HomePage, Header e Footer.

2.1.2. Application logic

La struttura del package si articola nei seguenti sub-package e classi:

- Package UtenteManagement
 - StoricoSegnalazioneServlet
 - LoginServlet
 - LogoutServlet
 - ModificaDatiServlet
 - RegistrazioneServlet
- Package NotiziaManagement
 - VisualizzaNotizieServlet
 - RicercaNotizieServlet
 - ModificaStatoServlet
 - RegistraNotiziaServlet
- Package SegnalazioneManagement
 - InviaSegnalazioneServlet

- GestioneSegnalazioneServlet

2.1.3. Storage

La struttura del package include le seguenti classi:

- AccountUser
- AccountUserDao
- Notizia
- NotiziaDao
- Segnalazione
- SegnalazioneDao
- StoricoSegnalazione
- StoricoSegnalazioneDao

3. CLASS INTERFACES

3.1. AccountUserDao

NOME CLASSE	AccountUserDao
DESCRIZIONE	Gestisce l'accesso e la gestione degli account nel database
doUpdateProfilo	Metodo per aggiornare i dati del profilo utente. context AccountUserDao::doUpdateProfilo(email: String, nome: String, cognome: String, telefono: String) pre: email <> null and nome <> null and cognome <> null and telefono <> null post: self.doRetrieveByKey(email).nome = nome and self.doRetrieveByKey(email).cognome = cognome and self.doRetrieveByKey(email).telefono = telefono
doSave	Metodo per registrare un nuovo account. context AccountUserDao::doSave(user: AccountUser) pre: user <> null and user.email <> null and user.password <> null and user.name <> null and user.surname <> null and user.number <> null post: self.doRetrieveByKey(user.email).email = user.email and self.doRetrieveByKey(user.email).password = user.password and self.doRetrieveByKey(user.email).name = user.name and self.doRetrieveByKey(user.email).surname = user.surname and self.doRetrieveByKey(user.email).number = user.number
doSaveAmministratore	Metodo per registrare un nuovo account amministratore

	context AccountUserDao::doSaveAmministratore(user: AccountUser) pre: user <> null and user.email <> null and user.password <> null and user.name <> null and user.surname <> null and user.number <> null and user.ruolo = 1 post: self.doRetrieveByKey(user.email).email = user.email and self.doRetrieveByKey(user.email).password = user.password and self.doRetrieveByKey(user.email).name = user.name and self.doRetrieveByKey(user.email).surname = user.surname and self.doRetrieveByKey(user.email).number = user.number and self.doRetrieveByKey(user.email).ruolo = 1
doRetrieveAll	Metodo che restituisce una lista contenete tutti gli account. context AccountUserDao::doRetrieveAll(): Collection(AccountUser) pre: true post: result->forAll(u u.email <> null and u.name <> null and u.surname <> null and u.number <> null)

3.2. *NotiziaDao*

NOME CLASSE	NotiziaDao
DESCRIZIONE	Si occupa dell'accesso e la gestione delle notizie nel database.
doRetrieveByKey	Metodo per recuperare una notizia tramite ID. context NotiziaDao::doRetrieveByKey(id: Integer) : Notizia pre: id <> null post: (result.id = id and result.titolo <> null and result.contenuto <> null and result.fonte <> null and result.data_pubblicazione <> null and result.stato_verifica <> null) or (result.id = null and result.titolo = null and result.contenuto = null and result.fonte = null and result.data_pubblicazione = null and result.stato_verifica = null)
doSave	Metodo per salvare una nuova notizia. context NotiziaDao::doSave(notizia: Notizia) pre: notizia <> null and notizia.titolo <> null and notizia.contenuto <> null and notizia.fonte <> null and notizia.data_pubblicazione <> null post: self.doRetrieveByKey(notizia.id).titolo = notizia.titolo and self.doRetrieveByKey(notizia.id).contenuto = notizia.contenuto and self.doRetrieveByKey(notizia.id).fonte = notizia.fonte and self.doRetrieveByKey(notizia.id).data_pubblicazione = notizia.data_pubblicazione
doUpdateStatoVerifica	Metodo per aggiornare lo stato di verifica di una notizia. context NotiziaDao::doUpdateStatoVerifica(id: Integer, stato: String) pre: id <> null and stato <> null and (stato = 'Verificata' or stato = 'In verifica' or stato = 'Non attendibile') post: self.doRetrieveByKey(id).stato_verifica = stato
doDelete	Metodo per eliminare una notizia. context NotiziaDao::doDelete(id: Integer) pre: id <> null post: Notizia.allInstances()->forAll(n n.id <> id)
doRetrieveAll	Metodo per recuperare tutte le notizie. context NotiziaDao::doRetrieveAll(): Collection(Notizia) pre: true post: result->forAll(n n.id <> null and n.titolo <> null and n.stato_verifica <> null)
doRetrieveByTitolo	Metodo per cercare notizie per titolo. context NotiziaDao::doRetrieveByTitolo(titolo: String): Collection(Notizia) pre: titolo <> null post: result->forAll(n n.titolo.toLowerCase().contains(titolo.toLowerCase()))

3.3. *SegnalazioneDao*

NOME CLASSE	SegnalazioneDao
DESCRIZIONE	Si occupa dell'accesso e la gestione delle segnalazioni nel database.
doSave	Metodo per salvare una nuova segnalazione. context SegnalazioneDao::doSave(segnalazione: Segnalazione) pre: segnalazione <> null and segnalazione.titolo <> null and segnalazione.motivazione <> null and segnalazione.link_fonte_affidabile <> null and segnalazione.data_segnalazione <> null and segnalazione.stato <> null post: self.doRetrieveByKey(segnalazione.id).titolo = segnalazione.titolo and self.doRetrieveByKey(segnalazione.id).motivazione = segnalazione.motivazione and self.doRetrieveByKey(segnalazione.id).link_fonte_affidabile = segnalazione.link_fonte_affidabile
doRetrieveByKey	Metodo per recuperare una segnalazione tramite ID. context SegnalazioneDao::doRetrieveByKey(id: Integer) : Segnalazione pre: id <> null post: (result.id = id and result.titolo <> null and result.motivazione <> null and result.data_segnalazione <> null and result.stato <> null) or (result.id = null and result.titolo = null and result.motivazione = null and result.data_segnalazione = null and result.stato = null)
doUpdateStato	Metodo per aggiornare lo stato di una segnalazione. context SegnalazioneDao::doUpdateStato(id: Integer, stato: String) pre: id <> null and stato <> null post: self.doRetrieveByKey(id).stato = stato
doRetrieveAll	Metodo per recuperare tutte le segnalazioni. context SegnalazioneDao::doRetrieveAll(): Collection(Segnalazione) pre: true post: result->forAll(s s.id <> null and s.titolo <> null and s.data_segnalazione <> null and s.stato <> null)
doRetrieveByUtente	Metodo per recuperare le segnalazioni di un utente specifico. context SegnalazioneDao::doRetrieveByUtente(email: String): Collection(Segnalazione) pre: email <> null post: result->forAll(s s.utente_email = email)

3.4. *StoricoSegnalazioneDao*

NOME CLASSE	StoricoSegnalazioneDao
DESCRIZIONE	Si occupa dell'accesso e la gestione dello storico delle segnalazioni.
doSave	Metodo per salvare una voce nello storico. context StoricoSegnalazioneDao::doSave(storico: StoricoSegnalazione) pre: storico <> null and storico.data_visualizzazione <> null post: self.doRetrieveByKey(storico.id).data_visualizzazione = storico.data_visualizzazione
doRetrieveByUtente	Metodo per recuperare lo storico di un utente. context StoricoSegnalazioneDao::doRetrieveByUtente(email: String): Collection(StoricoSegnalazione) pre: email <> null post: result->forAll(s s.utente_email = email)
doRetrieveByKey	Metodo per recuperare una voce di storico tramite ID. context StoricoSegnalazioneDao::doRetrieveByKey(id: Integer) : StoricoSegnalazione pre: id <> null post: (result.id = id and result.data_visualizzazione <> null) or (result.id = null and result.data_visualizzazione = null)